

Memory Organization and Memory Technology

Recap: ISA vs Microarchitecture

Computer Architecture

ISA vs. Microarchitecture

- What is part of ISA vs. Uarch?
 - Gas pedal: interface for “acceleration”
 - Internals of the engine: implement “acceleration”
- Implementation (uarch) can be various as long as it satisfies the specification (ISA)
 - Add instruction vs. Adder implementation
 - Bit serial, ripple carry, carry lookahead adders are all part of microarchitecture
 - x86 ISA has many implementations: 286, 386, 486, Pentium, Pentium Pro, Pentium 4, Core, ...
- Microarchitecture usually changes faster than ISA
 - Few ISAs (x86, ARM, SPARC, MIPS, Alpha) but many uarchs
 - *Why?*

ISA

- Instructions
 - Opcodes, Addressing Modes, Data Types
 - Instruction Types and Formats
 - Registers, Condition Codes
- Memory
 - Address space, Addressability, Alignment
 - Virtual memory management
- Interrupt/Exception Handling
- Access Control, Priority/Privilege
- I/O: memory-mapped vs. special instruction
- Task/thread Management
- Multi-threading support, Multiprocessor support

Effect of Translation

- One can translate from one ISA to another *ISA* to change the semantic gap tradeoffs
 - ISA (virtual ISA) → Implementation ISA
- Examples
 - Intel's and AMD's x86 implementations translate x86 instructions into programmer-invisible microoperations (simple instructions) in hardware
 - Transmeta's x86 implementations translated x86 instructions into "secret" VLIW instructions in software (code morphing software)

Software-Based Translation

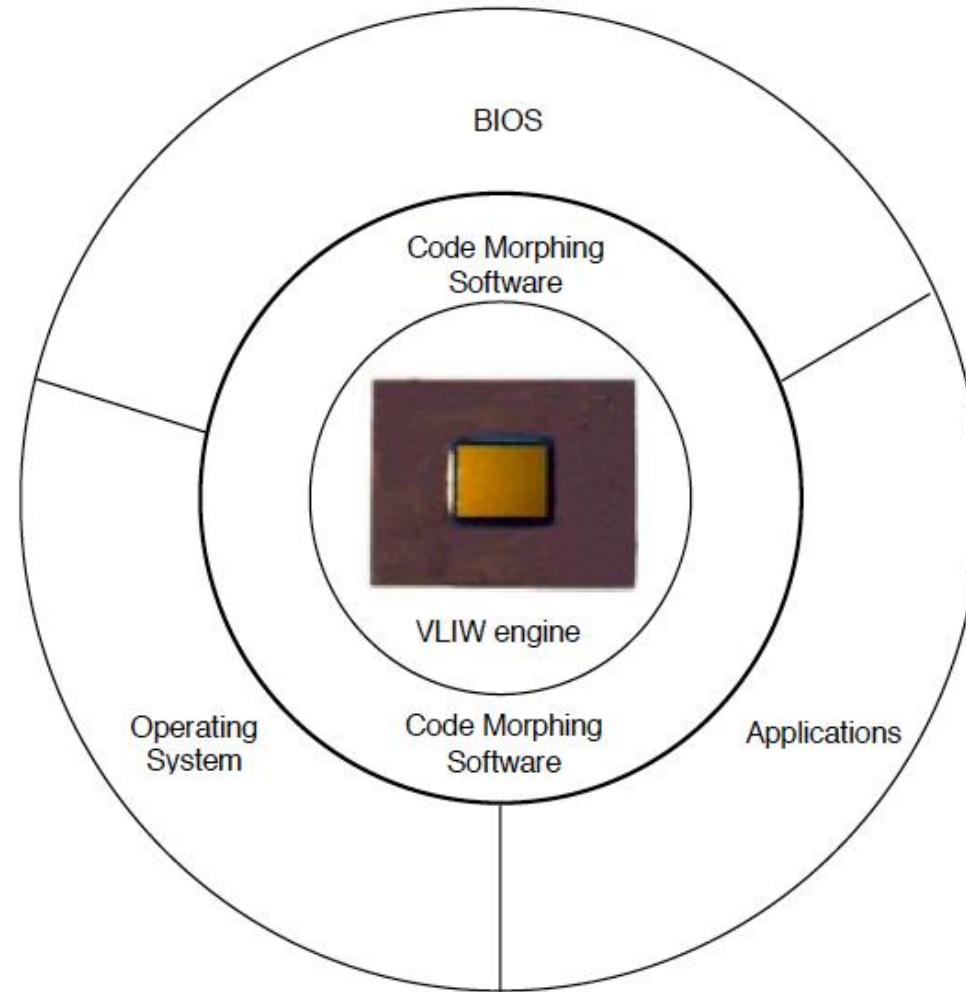


Figure 5. The Code Morphing software mediates between x86 software and the Crusoe processor.

Klaiber, [“The Technology Behind Crusoe Processors,”](#) Transmeta White Paper 2000.

ISA-level Tradeoffs: Number of Registers

- Affects:
 - Number of bits used for encoding register address
 - Number of values kept in fast storage (register file)
 - Size, access time, power consumption of register file
- Large number of registers:
 - + Enables better register allocation (and optimizations by compiler) → fewer saves/restores
 - Larger instruction size

ISA-level Tradeoffs: Addressing Modes

- Addressing mode specifies how to obtain an operand of an instruction
 - Register
 - Immediate
 - Memory (displacement, register indirect, indexed, absolute, memory indirect, autoincrement, autodecrement, ...)
- More modes:
 - + help better support programming constructs (arrays, pointer-based accesses)
 - make it harder for the architect to design
 - too many choices for the compiler?
 - Many ways to do the same thing complicates compiler design

Back to Programmer vs. (Micro)architect

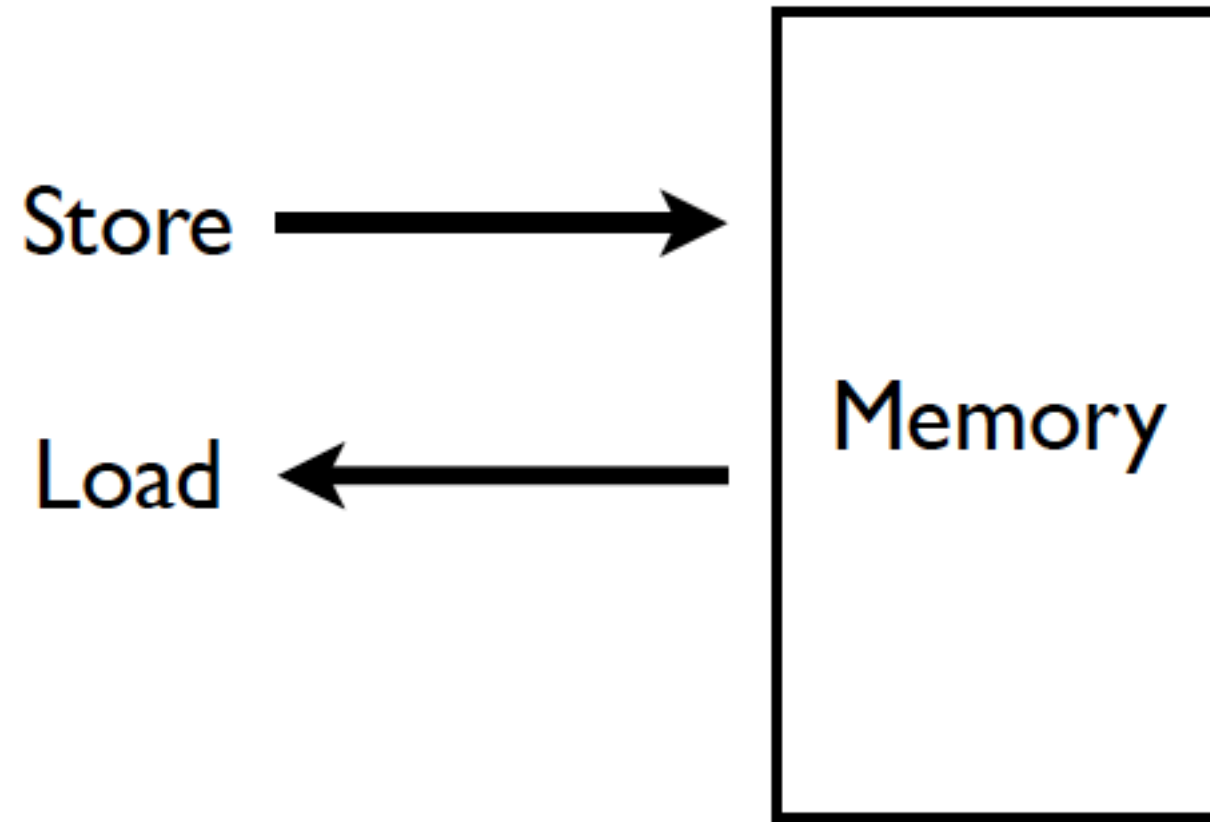
- Many ISA features designed to aid programmers
- But, complicate the hardware designer's job
- Virtual memory
 - vs. overlay programming
 - Should the programmer be concerned about the size of code blocks fitting physical memory?
- Addressing modes

Memory Organization and Memory Technology

Readings

- *Digital Design and Computer Architecture – David Harris & Sarah Harris*
 - Chapter 8

Memory (Programmer's View)



Abstraction: Virtual vs. Physical Memory

- **Programmer** sees **virtual memory**
 - Can assume the memory is "infinite"
 - Reality: **Physical memory** size is much smaller than what the programmer assumes
 - **The system** (system software + hardware, cooperatively) maps **virtual memory addresses** to **physical memory**
 - The system automatically manages the physical memory space **transparently to the programmer**
- + Programmer does not need to know the physical size of memory nor manage it →
A small physical memory can appear as a huge one to the programmer → Life is easier for the programmer
- More complex system software and architecture

A classic example of the programmer/(micro)architect tradeoff

(Physical) Memory System

- You need a larger level of storage to manage a small amount of physical memory automatically
→ Physical memory has a backing store: disk
- We will first start with the physical memory system
- For now, ignore the virtual→physical indirection
- We will get back to it later, if time permits...

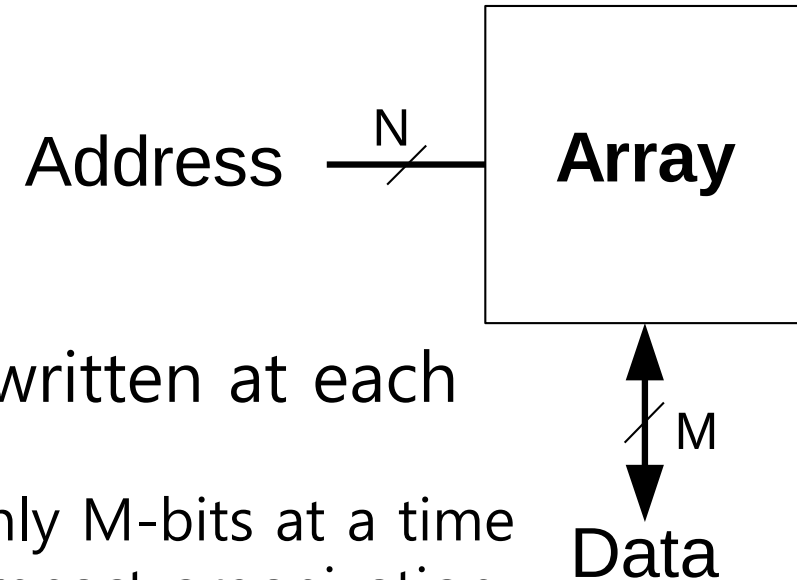
Quick Overview of Memory Arrays

How Can We Store Data?

- Flip-Flops (or Latches)
 - Very fast
 - Very expensive (one bit costs tens of transistors)
- Static RAM
 - Relatively fast, only one data word at a time
 - Expensive (one bit costs 6 transistors)
- Dynamic RAM
 - Slower, one data word at a time, needs periodic refresh
 - Cheap (one bit costs only one transistor plus one capacitor)
- Other storage technology (flash memory, hard disk, tape)
 - Much slower, access takes a long time, non-volatile
 - Very cheap (no transistors directly involved)

Array Organization of Memories

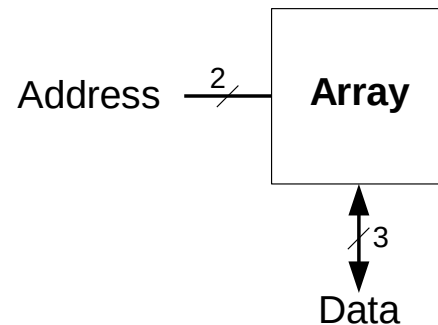
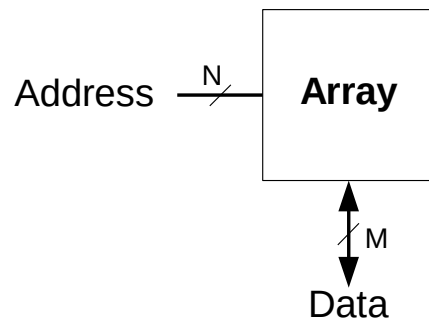
- Goal: Efficiently store large amounts of data
 - A memory array (stores data)
 - Address selection logic (selects one row of the array)
 - Readout circuitry (reads data out)



- An M-bit value can be read or written at each unique N-bit address
 - All values can be accessed, but only M-bits at a time
 - Access restriction allows more compact organization

Memory Arrays

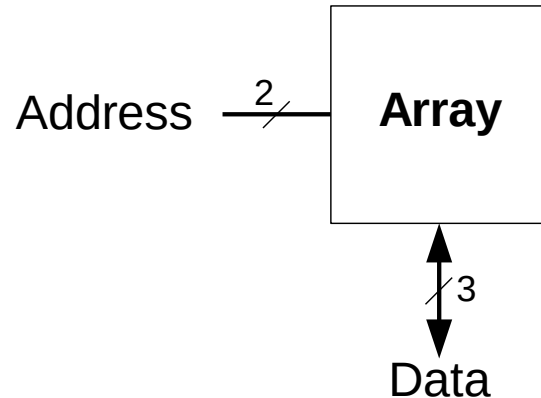
- Two-dimensional array of bit cells
 - Each bit cell stores one bit
- An array with N address bits and M data bits:
 - 2^N rows and M columns
 - Depth: number of rows (number of words)
 - Width: number of columns (size of word)
 - Array size: depth $\times$ width = $2^N \times M$



Address	Data			
11	0	1	0	depth ↑ ↓
10	1	0	0	
01	1	1	0	
00	0	1	1	
	width ←→			

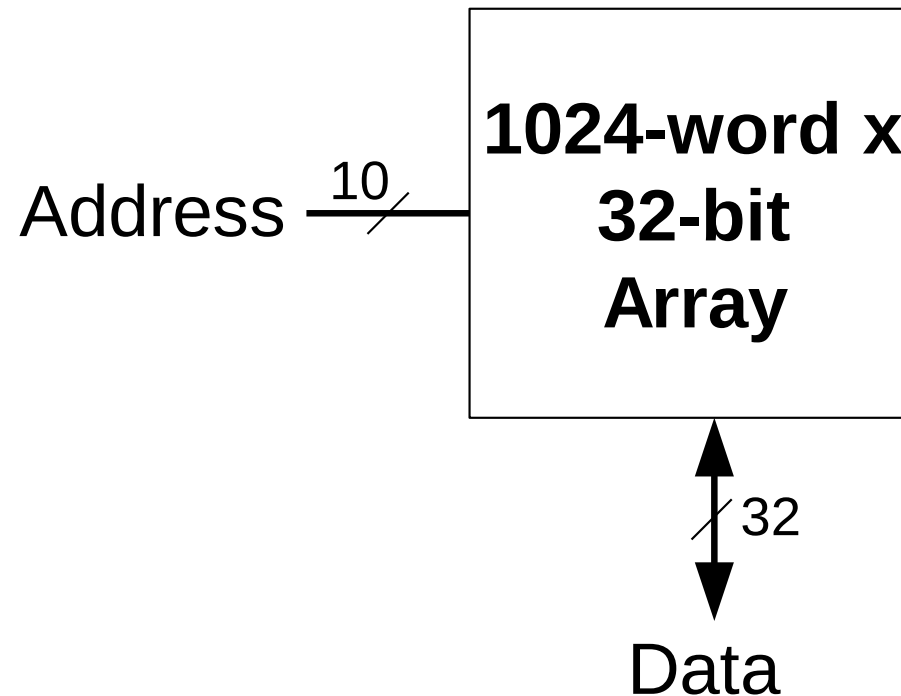
Memory Array Example

- $2^2 \times 3$ -bit array
- Number of words: 4
- Word size: 3-bits
- For example, the 3-bit word stored at address 10 is 100



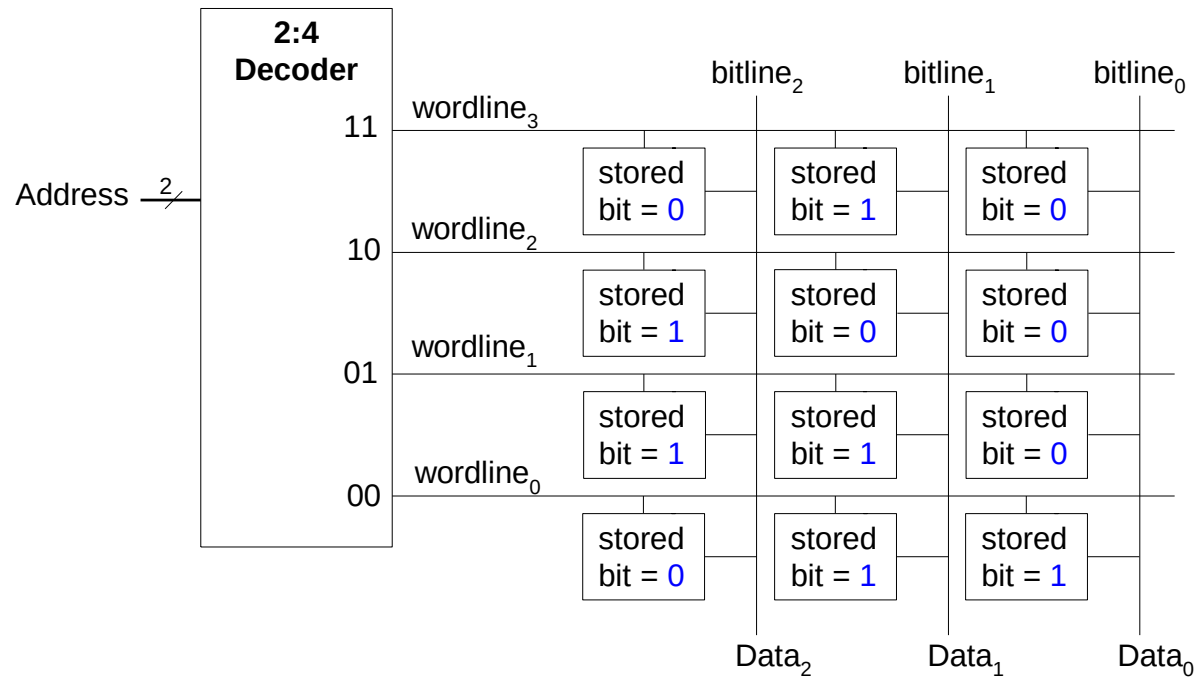
Address	Data			
11	0	1	0	depth ↑ ↓
10	1	0	0	
01	1	1	0	
00	0	1	1	
	width ←→			

Larger and Wider Memory Array Example



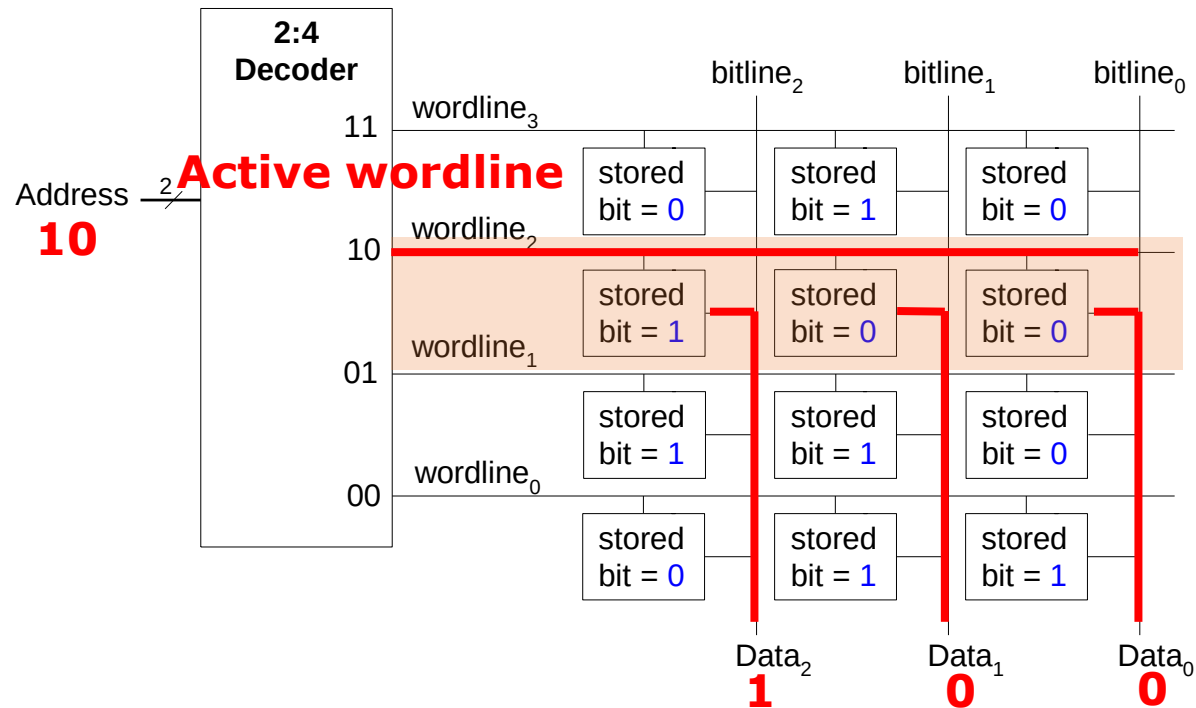
Memory Array Organization (I)

- Storage nodes in one column connected to one bitline
- Address decoder activates only ONE wordline
- Content of one line of storage available at output



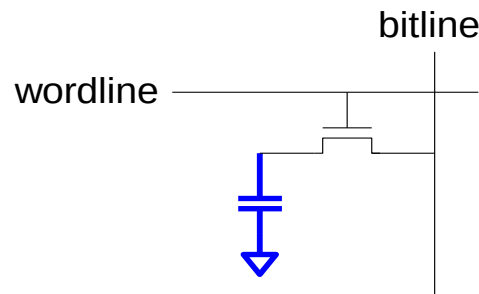
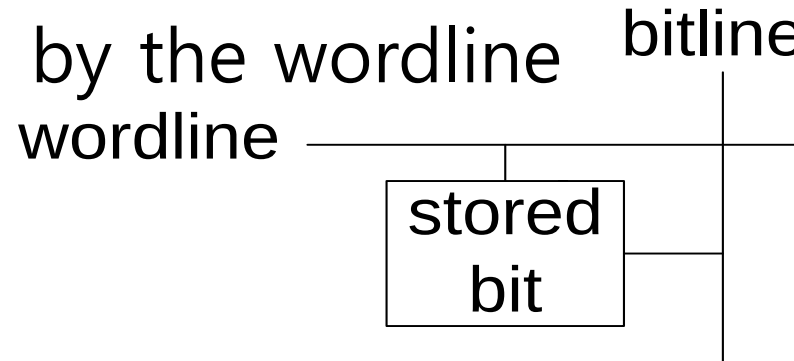
Memory Array Organization (II)

- Storage nodes in one column connected to one bitline
- Address decoder activates only ONE wordline
- Content of one line of storage available at output

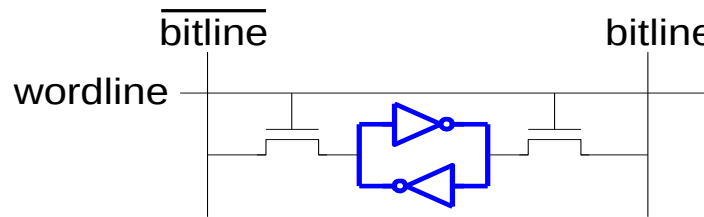


How is Access Controlled?

- Access transistors configured as switches connect the bit storage to the bitline.
- Access controlled by the wordline



DRAM



SRAM

Building Larger Memories

- Requires larger memory arrays
- Large → slow
- How do we make the memory large without making it very slow?
- Idea: Divide the memory into smaller arrays and interconnect the arrays to input/output buses
 - Large memories are hierarchical array structures
 - DRAM: Channel → Rank → Bank → Subarrays → Mats

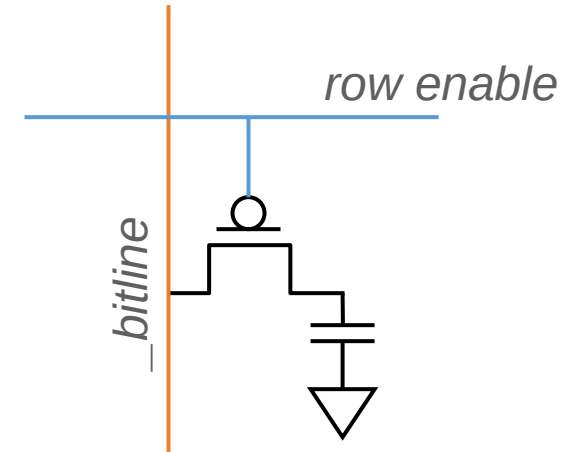
General Principle: Interleaving (Banking)

- **Interleaving (banking)**
 - **Problem:** a single monolithic large memory array takes long to access and does not enable multiple accesses in parallel
 - **Goal:** Reduce the latency of memory array access and enable multiple accesses in parallel
 - **Idea:** Divide a large array into multiple banks that can be accessed independently (in the same cycle or in consecutive cycles)
 - Each bank is smaller than the entire memory storage
 - Accesses to different banks can be overlapped
 - **A Key Issue:** How do you map data to different banks? (i.e., how do you interleave data across banks?)

Memory Technology: DRAM and SRAM

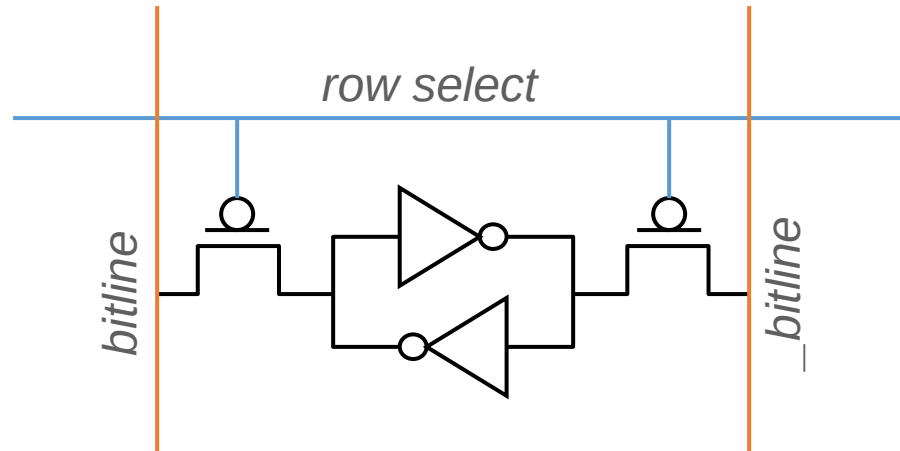
Memory Technology: DRAM

- Dynamic random access memory
- Capacitor charge state indicates stored value
 - Whether the capacitor is charged or discharged indicates storage of 1 or 0
 - 1 capacitor
 - 1 access transistor
- Capacitor leaks
 - DRAM cell loses charge over time
 - DRAM cell needs to be refreshed

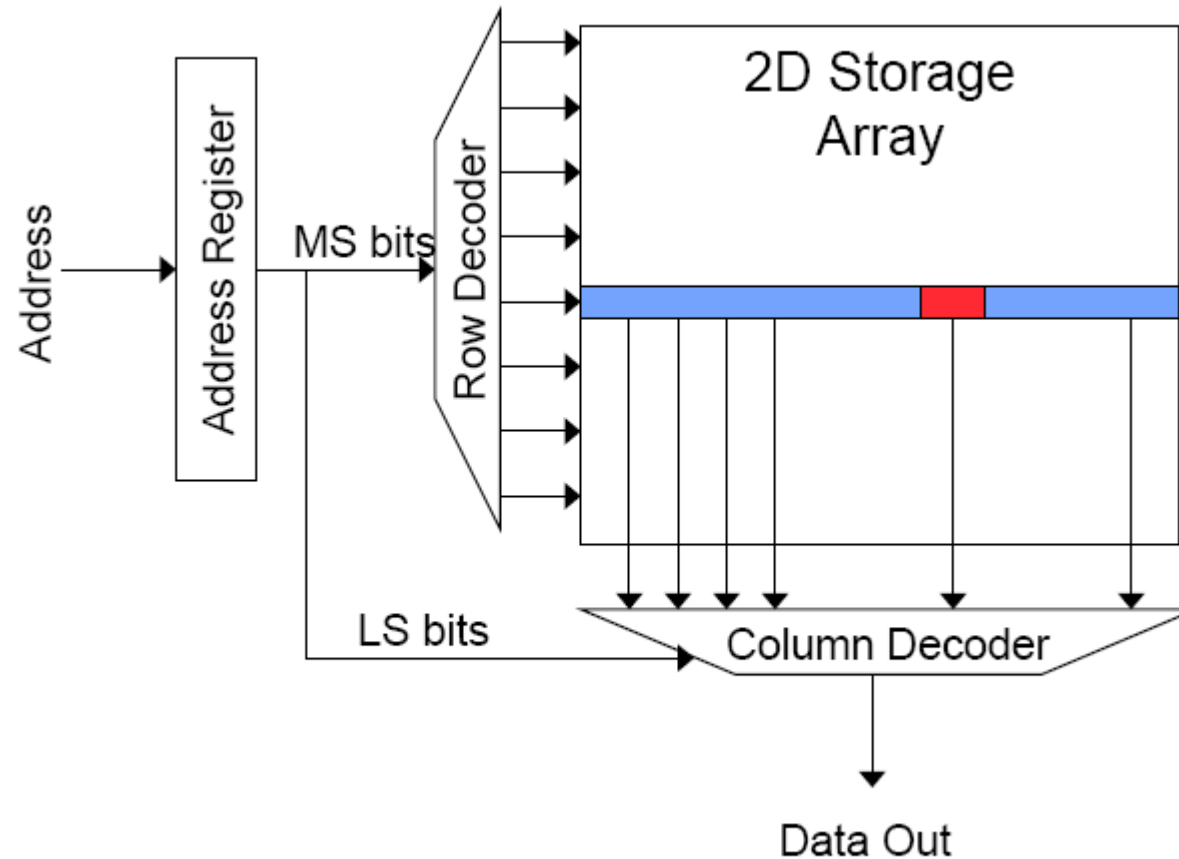


Memory Technology: SRAM

- Static random access memory
- Two cross coupled inverters store a single bit
 - Feedback path enables the stored value to persist in the “cell”
 - 4 transistors for storage
 - 2 transistors for access



Memory Bank Organization and Operation



- Read access sequence:

1. Decode row address & drive word-lines

2. Selected bits drive bit-lines

- Entire row read

3. Amplify row data

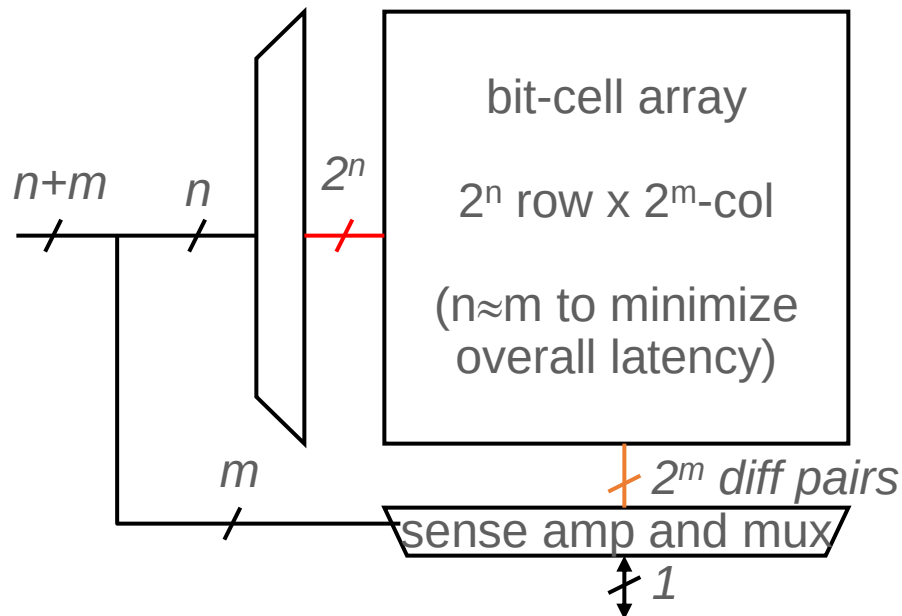
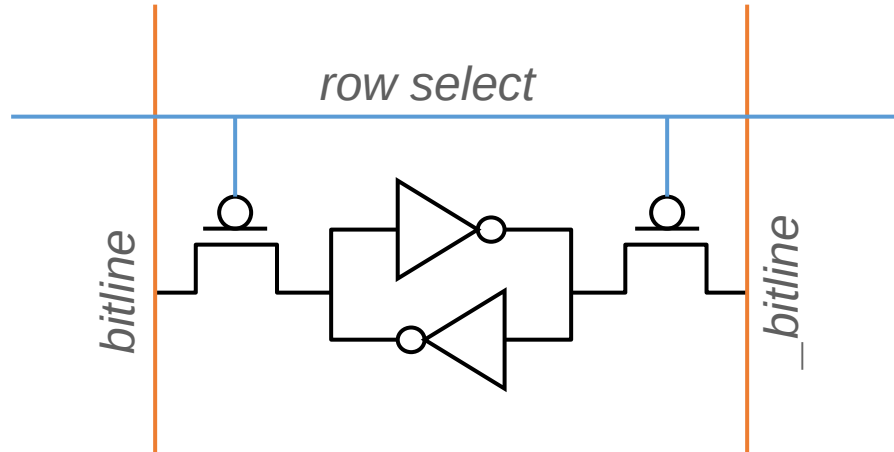
4. Decode column address & select subset of row

- Send to output

5. Precharge bit-lines

- For next access

SRAM (Static Random Access Memory)



Read Sequence

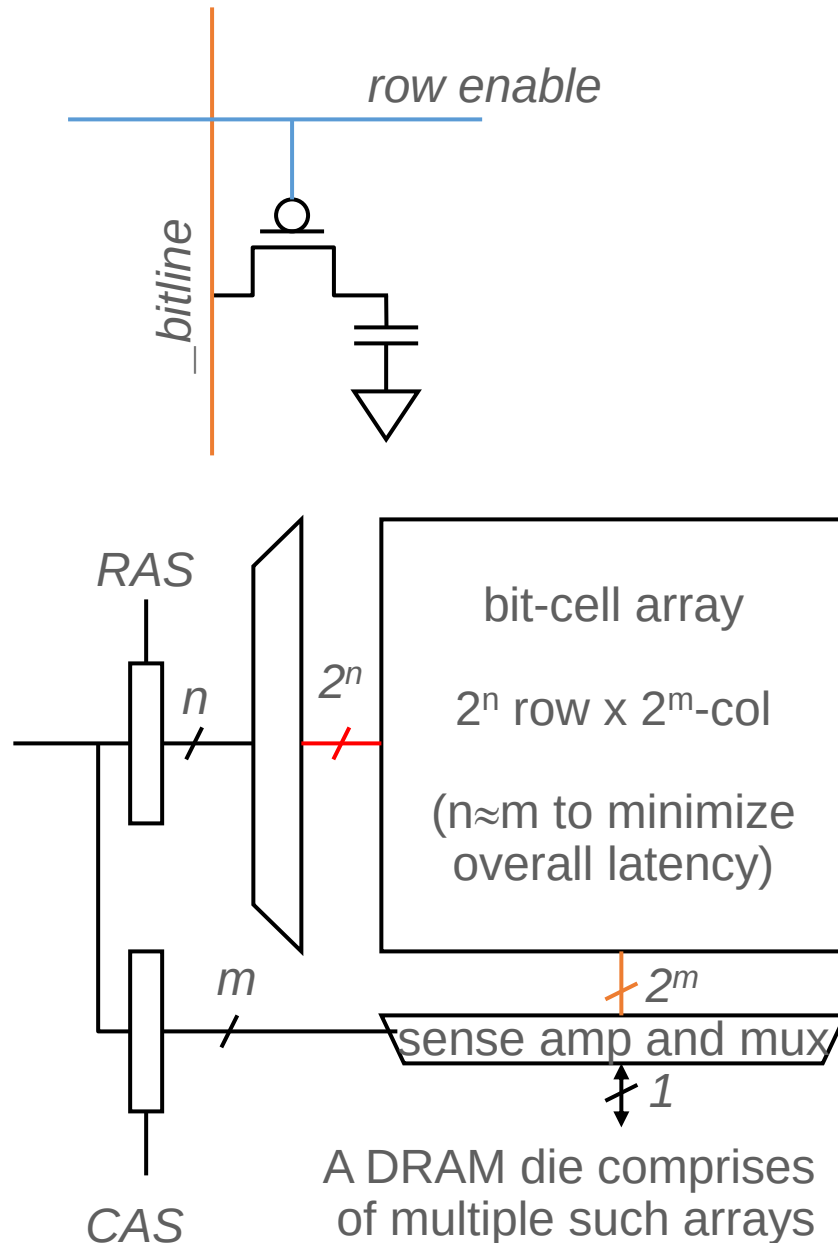
1. address decode
2. drive row select
3. selected bit-cells drive bitlines
(entire row is read together)
4. differential sensing and column select
(data is ready)
5. precharge all bitlines
(for next read or write)

Access latency dominated by steps 2 and 3

Cycling time dominated by steps 2, 3 and 5

- step 2 proportional to 2^m
- step 3 and 5 proportional to 2^n

DRAM (Dynamic Random Access Memory)



Bits stored as charges on node capacitance (non-restorative)

- bit cell loses charge when read
- bit cell loses charge over time

Read Sequence

1~3 same as SRAM

4. a “flip-flopping” sense amp amplifies and regenerates the bitline, data bit is mux’ed out

5. precharge all bitlines

Destructive reads

Charge loss over time

Refresh: A DRAM controller must periodically read each row within the allowed refresh time (10s of ms) such that charge is restored

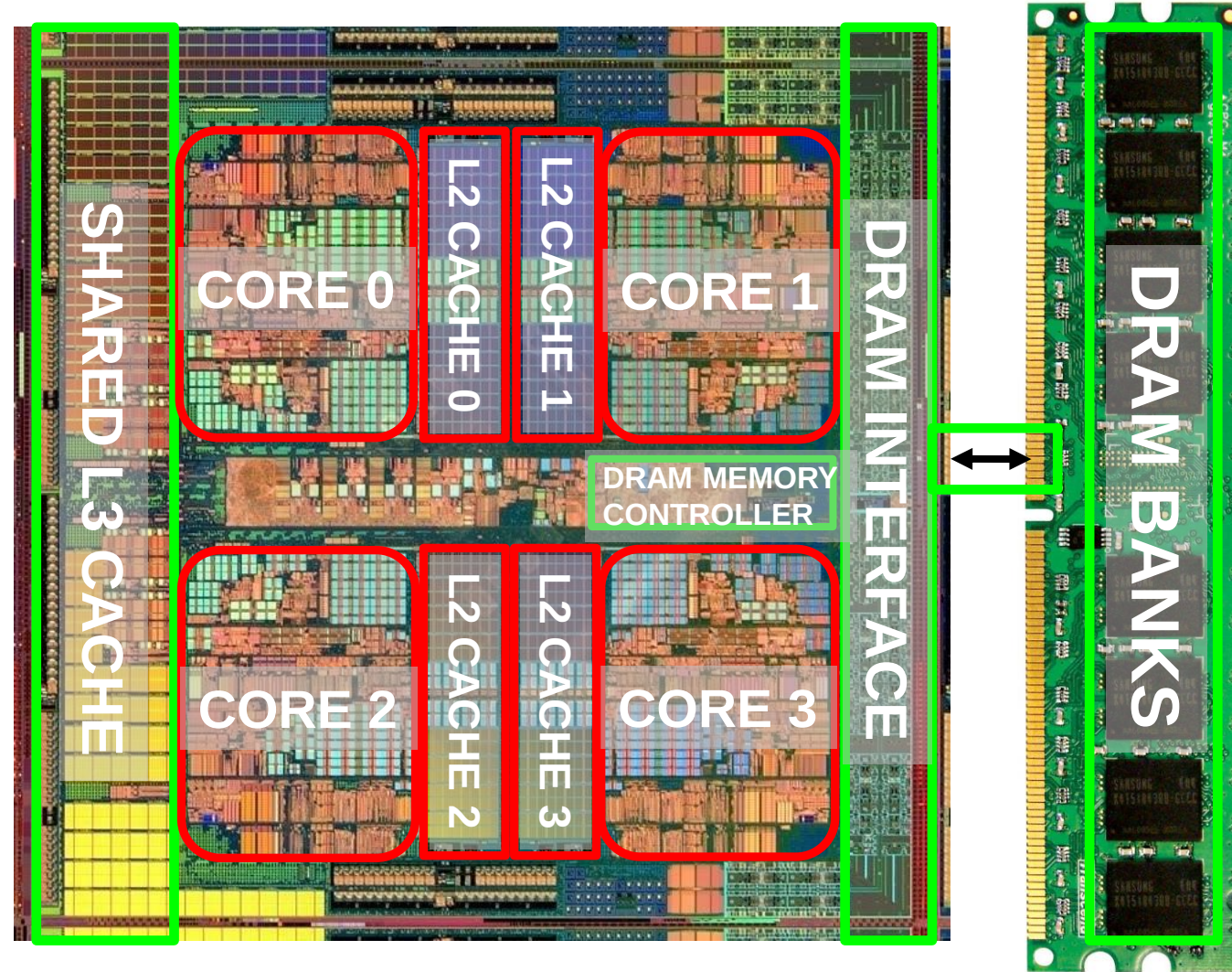
DRAM vs. SRAM

- DRAM
 - Slower access (capacitor)
 - Higher density (1T 1C cell)
 - Lower cost
 - Requires refresh (power, performance, circuitry)
 - Manufacturing requires putting capacitor and logic together
- SRAM
 - Faster access (no capacitor)
 - Lower density (6T cell)
 - Higher cost
 - No need for refresh
 - Manufacturing compatible with logic process (no capacitor)

Memory Hierarchy and Caches

The Memory Hierarchy

Memory in a Modern System



Ideal Memory

- Zero access time (latency)
- Infinite capacity
- Zero cost
- Infinite bandwidth (to support multiple accesses in parallel)

The Problem

- Ideal memory's requirements oppose each other
- Bigger is slower
 - Bigger → Takes longer to determine the location
- Faster is more expensive
 - Memory technology: SRAM vs. DRAM vs. Disk vs. Tape
- Higher bandwidth is more expensive
 - Need more banks, higher frequency, or faster technology

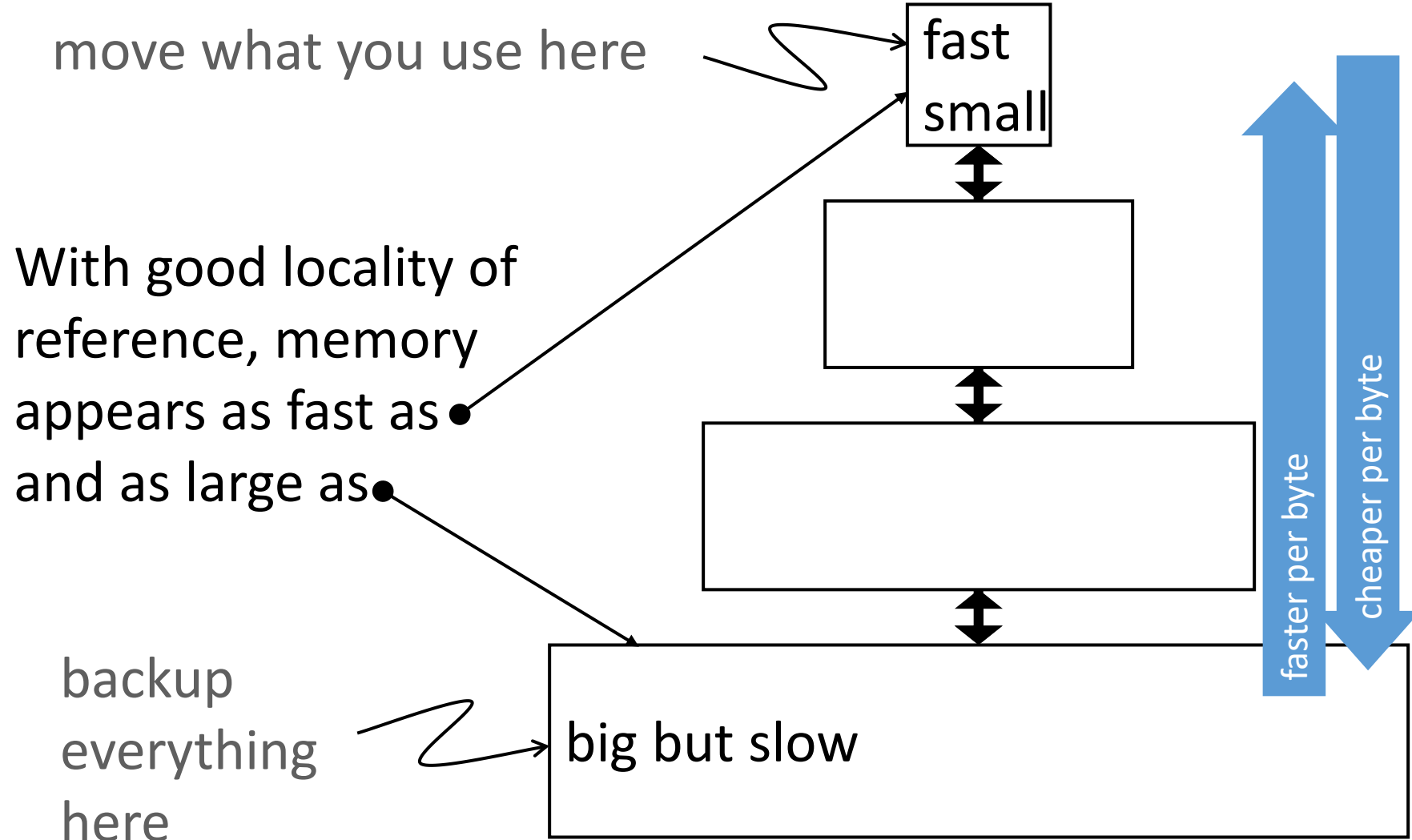
The Problem

- Bigger is slower
 - SRAM, 512 Bytes, sub-nanosec
 - SRAM, KByte~MByte, ~nanosec
 - DRAM, Gigabyte, ~50 nanosec
 - Hard Disk, Terabyte, ~10 millisec
- Faster is more expensive (dollars and chip area)
 - SRAM, < 10\$ per Megabyte
 - DRAM, < 1\$ per Megabyte
 - Hard Disk < 1\$ per Gigabyte
 - These sample values (circa ~2011) scale with time

Why Memory Hierarchy?

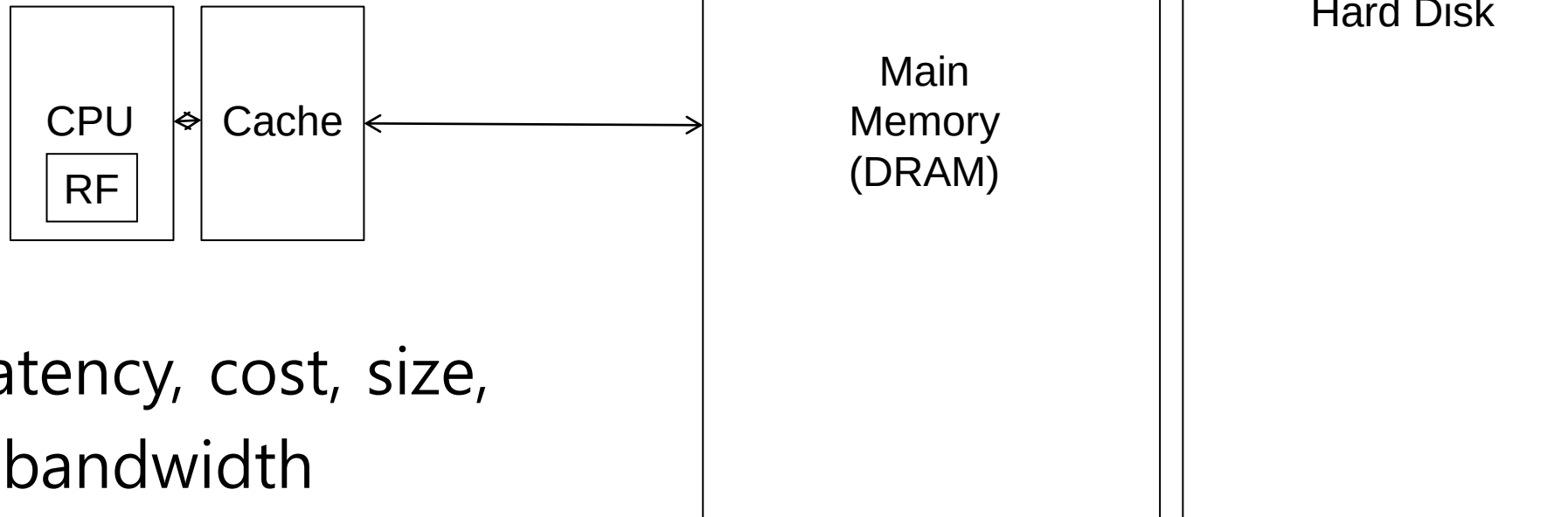
- We want both fast and large
- But we cannot achieve both with a single level of memory
- Idea: Have multiple levels of storage (progressively bigger and slower as the levels are farther from the processor) and ensure most of the data the processor needs is kept in the fast(er) level(s)

The Memory Hierarchy



Memory Hierarchy

- Fundamental tradeoff
 - Fast memory: small
 - Large memory: slow
- Idea: **Memory hierarchy**



- Latency, cost, size, bandwidth

Locality

- One's recent past is a very good predictor of his/her near future.
- **Temporal Locality**: If you just did something, it is very likely that you will do the same thing again soon
 - since you are here today, there is a good chance you will be here again and again regularly
- **Spatial Locality**: If you did something, it is very likely you will do something similar/related (in space)
 - every time I find you in this room, you are probably sitting close to the same people

Memory Locality

- A “typical” program has a lot of locality in memory references
 - typical programs are composed of “loops”
- **Temporal**: A program tends to reference the same memory location many times and all within a small window of time
- **Spatial**: A program tends to reference a cluster of memory locations at a time
 - most notable examples:
 - 1. instruction memory references
 - 2. array/data structure references

Caching Basics: Exploit Temporal Locality

- Idea: Store recently accessed data in automatically managed fast memory (called cache)
- Anticipation: the data will be accessed again soon
- Temporal locality principle
 - Recently accessed data will be again accessed in the near future
 - This is what Maurice Wilkes had in mind:
 - Wilkes, “Slave Memories and Dynamic Storage Allocation,” IEEE Trans. On Electronic Computers, 1965.
 - “The use is discussed of a fast core memory of, say 32000 words as a slave to a slower core memory of, say, one million words in such a way that in practical cases the effective access time is nearer that of the fast memory than that of the slow memory.”

Caching Basics: Exploit Spatial Locality

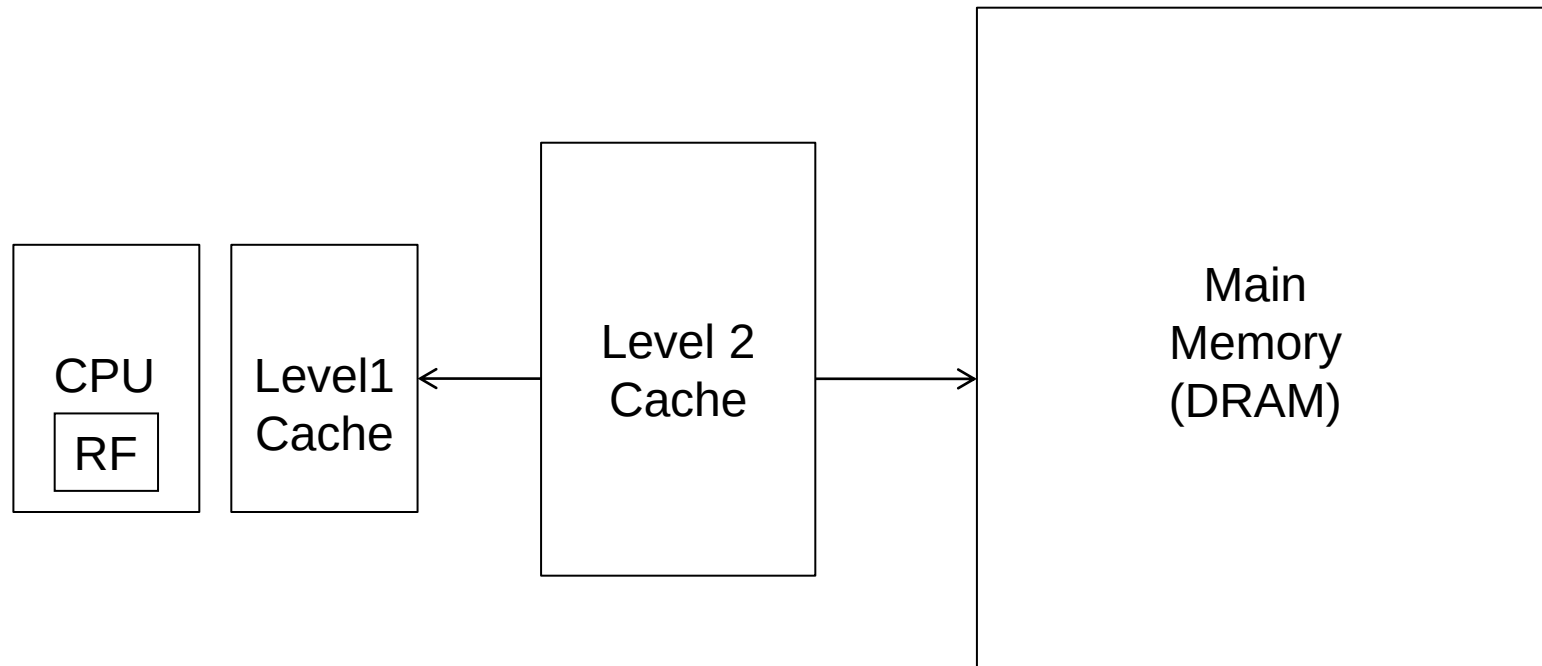
- Idea: Store addresses adjacent to the recently accessed one in automatically managed fast memory
 - Logically divide memory into equal size blocks
 - Fetch to cache the accessed block in its entirety
- Anticipation: nearby data will be accessed soon
- Spatial locality principle
 - Nearby data in memory will be accessed in the near future
 - E.g., sequential instruction access, array traversal
 - This is what IBM 360/85 implemented
 - 16 Kbyte cache with 64 byte blocks
 - Liptay, “Structural aspects of the System/360 Model 85 II: the cache,” IBM Systems Journal, 1968.

The Bookshelf Analogy

- Book in your hand
- Desk
- Bookshelf
- Boxes at home
- Boxes in storage
- Recently-used books tend to stay on desk
 - Comp Arch books, books for classes you are currently taking
 - Until the desk gets full
- Adjacent books in the shelf needed around the same time
 - If I have organized/categorized my books well in the shelf

Caching in a Pipelined Design

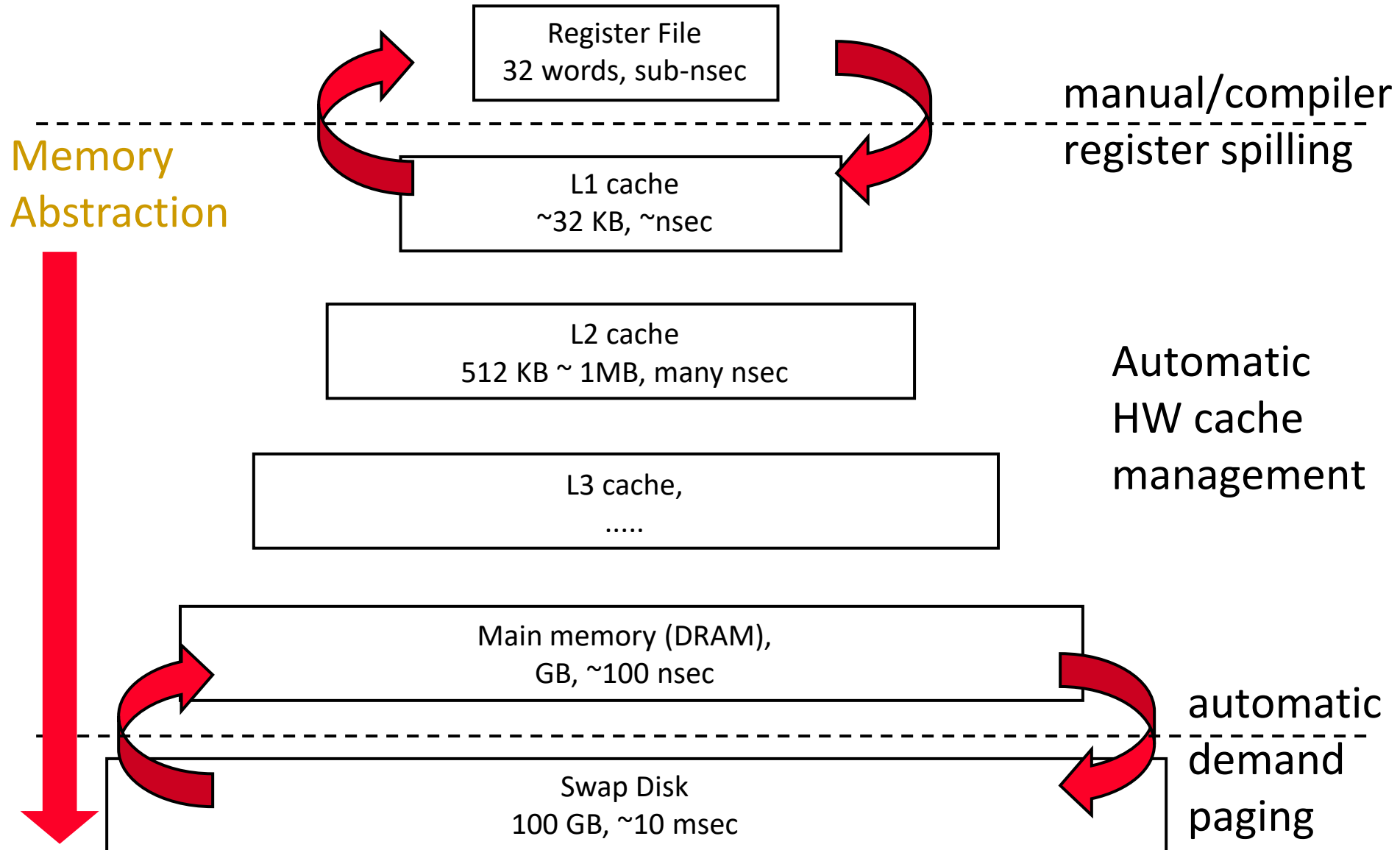
- The cache needs to be tightly integrated into the pipeline
 - Ideally, access in 1-cycle so that load-dependent operations do not stall
- High frequency pipeline → Cannot make the cache large
 - But, we want a large cache AND a pipelined design
- Idea: **Cache hierarchy**



A Note on Manual vs. Automatic Management

- **Manual:** Programmer manages data movement across levels
 - too painful for programmers on substantial programs
 - still done in some embedded processors (on-chip scratch pad SRAM in lieu of a cache) and GPUs (called "shared memory")
- **Automatic:** Hardware manages data movement across levels, transparently to the programmer
 - ++ programmer's life is easier
 - the average programmer doesn't need to know about it
 - You don't need to know how big the cache is and how it works to write a "correct" program! (What if you want a "fast" program?)

A Modern Memory Hierarchy



Hierarchical Latency Analysis

- For a given memory hierarchy level i it has a technology-intrinsic access time of t_i . The perceived access time T_i is longer than t_i
- Except for the outer-most hierarchy, when looking for a given address there is
 - a chance (hit-rate h_i) you “hit” and access time is t_i
 - a chance (miss-rate m_i) you “miss” and access time $t_i + T_{i+1}$
 - $h_i + m_i = 1$
- Thus

$$T_i = h_i \cdot t_i + m_i \cdot (t_i + T_{i+1})$$

$$T_i = t_i + m_i \cdot T_{i+1}$$

h_i and m_i are defined to be the hit-rate and miss-rate of just the references that missed at L_{i-1}

Hierarchy Design Considerations

- Recursive latency equation

$$T_i = t_i + m_i \cdot T_{i+1}$$

- The goal: achieve desired T_1 within allowed cost
- $T_i \approx t_i$ is desirable
- Keep m_i low
 - increasing capacity C_i lowers m_i , but beware of increasing t_i
 - lower m_i by smarter cache management (replacement::anticipate what you don't need, prefetching::anticipate what you will need)
- Keep T_{i+1} low
 - faster lower hierarchies, but beware of increasing cost
 - introduce intermediate hierarchies as a compromise

Intel Pentium 4 Example

- P4, 3.6 GHz
 - L1 cache
 - $C_1 = 16K$
 - $t_1 = 4$ cyc int / 9 cycle fp
 - L2 cache
 - $C_2 = 1024$ KB
 - $t_2 = 18$ cyc int / 18 cyc fp
 - Main memory
 - $t_3 = \sim 50ns$ or 180 cyc
 - Notice
 - best case latency is not 1
 - worst case access latencies are into 200+ cycles
- if $m_1=0.1, m_2=0.1$
 $T_1=7.6, T_2=36$
- if $m_1=0.01, m_2=0.01$
 $T_1=4.2, T_2=19.8$
- if $m_1=0.05, m_2=0.01$
 $T_1=5.00, T_2=19.8$
- if $m_1=0.01, m_2=0.50$
 $T_1=5.08, T_2=108$